

HBTXR: Algorithm–Hardware Co-Designed Hybrid Eye Tracking for On-Device Extended Reality

Anonymous Author(s)

Abstract—Real-time eye tracking on extended reality (XR) devices must operate under tight latency, energy, memory, and form-factor constraints while preserving robust pupil localization and gaze estimation. Prior work has mainly optimized either algorithms or accelerators in isolation, which leads to mismatches between model behavior and deployment cost. We present HBTXR, an algorithm–hardware co-designed hybrid eye-tracking framework that unifies frame-guided Search, event-guided Track, runtime scheduling, and accelerator mapping in one system pipeline. At the algorithm level, HBTXR introduces a time-synchronous and geometry-stable hybrid supervision flow, a modality-aware transformer with decoupled Search, Event, Track, and Mask heads, and a runtime-aware Search/Track scheduler. At the hardware level, HBTXR adopts quantization-aware and approximation-friendly operator design together with a mode-aware FPGA architecture composed of shared transformer processing, streaming feature delivery, and host-side scheduling control. Experimental results show that HBTXR achieves a pupil-center error of 0.42 pixels, a gaze error of 0.68° , and an end-to-end Track-mode latency of 0.57 ms, while maintaining 97.8% Search success rate and 1754 Hz effective update rate in Track mode. These results suggest that HBTXR offers a practical system point for accurate, low-latency, and deployment-oriented XR eye tracking.

Index Terms—Eye tracking, extended reality, hybrid sensing, event camera, transformer accelerator, FPGA, algorithm–hardware co-design.

I. INTRODUCTION

Eye tracking has become a core enabling function for extended reality (XR), supporting gaze-based interaction, foveated rendering, user-state understanding, and privacy-preserving on-device perception. In practice, however, XR eye tracking must satisfy three requirements simultaneously: it must remain semantically robust under appearance changes, temporally responsive under rapid eye motion, and efficient enough to run on embedded hardware with strict power and memory budgets. This combination makes XR eye tracking fundamentally a system-level design problem rather than a stand-alone perception problem.

Existing approaches reveal a structural trade-off. Frame-based methods are strong in semantic recovery and calibration-aware regression, but repeated dense-image processing introduces high bandwidth and latency cost. Event-based methods offer sparse sensing and high temporal resolution, but their reliability degrades under weak-event regimes, drift accumulation, or re-initialization failures. Hybrid methods improve this balance, yet many still treat fusion, runtime mode switching, and hardware deployment as loosely coupled stages.

This paper addresses that gap through HBTXR, an algorithm–hardware co-designed hybrid eye-tracking system for on-device

XR. In contrast to monolithic multimodal regression, HBTXR explicitly separates *Search* and *Track*. Search is frame-guided and refresh-oriented, designed for robust relocalization and anchor recovery. Track is event-guided and state-persistent, designed for low-latency residual update around the current pupil estimate. This decomposition is reflected not only in the model architecture, but also in the supervision flow, training pipeline, runtime scheduler, and hardware architecture.

Fig. 1 summarizes the overall design. HBTXR first constructs time-synchronous supervision so that frame and event inputs share one stable ellipse-centric target. It then employs a modality-aware transformer with a shared backbone and decoupled heads. A runtime scheduler switches between Search and Track according to confidence, similarity, event density, and eye-state validity. Finally, a CPU–FPGA deployment architecture maps the shared backbone and lightweight heads to differentiated execution paths. In this way, HBTXR follows the AHCO-style principle of internalizing hardware constraints during algorithm design while tuning the hardware around the actual runtime semantics of the algorithm.

The key contributions of this work are summarized as follows.

- We reformulate hybrid eye tracking as a Search–Track problem and introduce a time-synchronous, geometry-stable supervision contract for aligning frame, event, and state targets.
- We design a modality-aware transformer with shared backbone and decoupled heads for Search, Event, Track, and Mask prediction.
- We develop a runtime-aware scheduler that explicitly manages Search mode, Track mode, and mode switching according to quality assessment signals.
- We present a deployment-oriented accelerator design with quantization-aware operator reformulation, mixed-granularity execution, streaming feature delivery, and CPU–FPGA co-scheduling.

II. RELATED WORKS

A. Frame-based Eye Tracking

Frame-based eye tracking remains the dominant paradigm in near-eye and head-mounted systems because grayscale or infrared frames preserve strong semantic structure for pupil localization, iris fitting, and gaze regression. These methods are particularly effective for global relocalization and calibration-aware estimation. However, their strengths come with dense sensing and repeated full-frame processing, which translate

Figure placeholder
Final artwork pending

Fig. 1. Overview of HBTXR. The proposed framework combines time-synchronous hybrid supervision, a modality-aware Search/Track transformer, a runtime-aware scheduler, and a deployment-oriented FPGA architecture.

into high memory traffic, bandwidth overhead, and limited responsiveness under rapid eye motion.

B. Event-based Eye Tracking

Event-based eye tracking exploits asynchronous brightness-change sensing to achieve sparse and high-temporal-resolution processing. Prior work has shown that event-driven methods are particularly attractive for low-latency update during fast saccades and for reducing redundant processing during fixation. Nonetheless, event-only pipelines remain weaker in semantic stabilization, blink recovery, and long-horizon relocalization, especially when event density becomes low or the tracker drifts away from the valid pupil contour.

C. Hybrid Frame-Event Eye Tracking

Hybrid eye tracking combines frame-based semantic recovery with event-based temporal responsiveness. Representative systems demonstrate that relocalization can be executed on low-rate frames while event streams support high-rate update. This direction is highly promising for XR, but many prior hybrid systems still optimize the model and the deployment stack separately. As a result, Search and Track often remain implicit behaviors instead of explicit operating modes with distinct dataflows and control semantics.

D. Deployment-Oriented and Hardware Accelerator

A separate line of work targets embedded deployment and accelerator design. Frame-oriented systems such as EyeCoD focus on optics–algorithm–accelerator integration, whereas event-driven systems such as SEE and JaneEye explore FPGA or ASIC acceleration for sparse eye tracking. These works show that memory access pattern, synchronization overhead, workload orchestration, and on-chip buffering are as important as raw arithmetic throughput. In parallel, transformer accelerators such as HG-PIPE illustrate how mixed-granularity pipeline design and non-linear operator approximation can greatly improve deployment efficiency for attention-based models.

E. Summary

The literature suggests four takeaways. First, frame cues are indispensable for robust recovery. Second, event cues are indispensable for low-latency update. Third, hybrid processing should explicitly distinguish refresh-oriented and reuse-oriented operation. Fourth, deployment efficiency improves when model structure and hardware dataflow are designed jointly. HBTXR is positioned at this intersection: it is neither a frame-only pipeline, nor an event-only pipeline, nor a loosely fused hybrid model. Instead, it is a runtime-aware and deployment-aware Search/Track system co-designed across supervision, architecture, scheduling, and hardware.

III. PROPOSED HBTXR ALGORITHM

A. Framework Design

1) *Problem Formulation*: HBTXR operates on a hybrid sensing input composed of a frame stream, an event stream, and a recurrent state. Let $I_t \in \mathbb{R}^{H \times W}$ denote the frame captured at time t , and let $E_t = \{e_k = (x_k, y_k, p_k, \tau_k)\}_{k=1}^{N_t}$ denote the event set within the current window, where (x_k, y_k) is the event coordinate, $p_k \in \{-1, +1\}$ is the polarity, and τ_k is the timestamp. The event stream is transformed into a polarity-split tensor $V_t \in \mathbb{R}^{2 \times H \times W}$ through a causal accumulation operator.

Eye Model: The target pupil state is parameterized as

$$\mathbf{s}_t = [x_t, y_t, a_t, b_t, u_t, v_t] \in \mathbb{R}^6, \quad (1)$$

where (x_t, y_t) is the pupil center, (a_t, b_t) are the ellipse axes, and $[u_t, v_t] = [\sin(2\theta_t), \cos(2\theta_t)]$ is the trigonometric rotation encoding. This representation avoids discontinuity around the angular wrap-around boundary and supports stable regression of ellipse orientation.

Objectness: Rather than only predicting ellipse parameters, HBTXR also models *objectness*, namely the likelihood that the current observation contains a valid and trackable pupil hypothesis. In Search mode, objectness is interpreted as a refresh confidence that indicates whether a frame-guided candidate should become the new anchor. In Track mode, objectness is paired with track confidence and track quality to assess whether the current event-driven residual update is still reliable. This objectness formulation becomes the interface between perception and scheduling.

2) *Framework Overview*: The overall framework consists of four interacting components: a supervision pipeline, a modality-aware transformer, a Search/Track scheduler, and a deployment path. Search performs frame-guided relocalization and anchor refresh. Track performs event-guided residual update conditioned on the previous state. An auxiliary event-side hypothesis is also estimated to measure cross-branch consistency. This explicit separation allows HBTXR to treat hybrid eye tracking as a system with two operating modes rather than as one undifferentiated multimodal regressor.

B. Time-Synchronous and Geometry-Stable Hybrid Supervision

Dataset Problem: A major difficulty in hybrid eye tracking is that frame labels are usually sparse in time whereas event data is asynchronous and dense. If the supervision timestamps are not aligned carefully, the model is forced to learn inconsistent geometry across frame, event, and recurrent inputs. HBTXR addresses this issue by constructing time-synchronous supervision around one canonical target representation.

1) *Frame Interpolation (TimeLens)*: To densify supervision in time, HBTXR first uses frame interpolation to synthesize intermediate frame observations between sparsely labeled frames. We describe this stage as a TimeLens-style interpolation block, whose purpose is not to create a new inference modality at runtime, but to construct better-aligned supervisory frames offline. The interpolated frames reduce the temporal gap between event windows and annotation anchors, which is particularly important during rapid saccades.

2) *Dense Annotation (Grounded-SAM)*: After temporal densification, HBTXR applies a dense annotation stage that refines eye-region and pupil masks from the interpolated frames. In the requested AHCO-style organization, this stage is expressed as a Grounded-SAM-assisted pseudo-labeling pipeline. The resulting masks are then converted to ellipse-centric annotations and validity metadata, including mask confidence, closed-eye flags, and track-valid indicators.

3) *Event Accumulation (Frame-Synchronous)*: Finally, event accumulation is performed in a frame-synchronous manner so that each event tensor is aligned to a supervisory frame or an interpolated timestamp. This step avoids ambiguous many-to-one mapping between event slices and frame labels. The output of the whole supervision pipeline is a canonical sample

$$(\bar{I}_t, \bar{V}_t, \bar{s}_{t-1}, \bar{s}_t, \bar{M}_t, \bar{B}_t, \bar{\xi}_t), \quad (2)$$

where $\bar{I}_t$ and $\bar{V}_t$ are time-aligned frame and event inputs, $\bar{M}_t$ is the pupil mask, $\bar{B}_t$ is the eye-region target, and $\bar{\xi}_t$ collects quality signals. This contract stabilizes geometry across modalities and propagates quality-aware supervision into both training and scheduling.

C. Modality-aware Transformer Architecture

1) *Front-end (Patch Embedding)*: HBTXR uses modality-specific front-ends instead of naïve early fusion. The canonical frame $\bar{I}_t$ is converted into frame tokens by a frame patch embedding module $P_f(\cdot)$ followed by a modality adapter $A_f(\cdot)$. Similarly, the canonical event tensor $\bar{V}_t$ is converted into event tokens by an event patch embedding module $P_e(\cdot)$ followed by

Figure placeholder
Final artwork pending

Fig. 2. Architecture of the proposed HBTXR algorithm. Frame and event inputs are processed by modality-specific patch embedding paths, a shared transformer backbone, and decoupled heads.

$A_e(\cdot)$. This separation lets the model preserve modality-specific statistics while still preparing both branches for shared token processing.

2) *Backbone (MHA, MLP)*: Both branches share a common transformer backbone composed of multi-head attention (MHA) and multi-layer perceptron (MLP) blocks. The shared backbone is intentionally partial rather than fully duplicated or fully unified at the pixel level. This allows Search-oriented frame inference and Track-oriented event inference to reuse the dominant representation trunk while keeping the front-end and head behavior specialized. The shared backbone also becomes the central object of hardware acceleration in the proposed FPGA design.

3) *Back-end (Head)*: On top of the shared backbone, HBTXR uses decoupled heads: an Eye Region Head, a Pupil Search Head, an Event Search Head, a Pupil Track Head, a Search Mask Head, and an Auxiliary State Head. Search heads estimate objectness and ellipse state from frame-guided features. Track heads regress residual motion from event features and previous state encoding. The Mask head acts as geometric regularization and an additional quality cue. This decoupled back-end is essential for preserving functional specialization under a shared backbone.

D. Runtime-aware Search and Track Scheduler

Search Mode: Search mode is refresh-oriented. It is activated at initialization or whenever the state becomes unreliable. In this mode, the system prioritizes semantic robustness and uses frame-guided inference to estimate a fresh pupil anchor, eye-region context, and mask-aware quality cues.

Track Mode: Track mode is state-persistent and low latency. Instead of solving the whole localization problem from scratch, it estimates a residual update around the previous state. Let $\psi(\bar{s}_{t-1})$ be the encoded previous state. The Track head predicts

$$\Delta \hat{\mathbf{r}}_t = h_t([\mathbf{p}_t^e; \psi(\bar{s}_{t-1})]), \quad (3)$$

where the residual vector contains center shift, axis-scale update, orientation update, track confidence, and track quality. The decoded state follows the residual update equations used in the current HBTXR draft.

Mode Switching (Quality Assessment): The scheduler decides between Search and Track according to multiple signals: Search confidence c_t^s , Track confidence c_t^{trk} , Track quality

q_t^{trk} , cross-branch similarity ρ_t , event density d_t , and eye-state indicator o_t . The transition rules are

$$\text{Search} \rightarrow \text{Track if } c_t^s > \tau_s, \quad (4)$$

$$\text{Track} \rightarrow \text{Search if } (c_t^{trk} \leq \tau_t) \vee (q_t^{trk} \leq \tau_q) \vee (\rho_t \leq \tau_\rho) \vee (d_t \leq \tau_d) \vee (o_t \leq \tau_o). \quad (5)$$

This quality assessment mechanism is central to the co-design philosophy because it not only improves tracking stability but also defines the control semantics exposed to the hardware scheduler.

E. Training Pipeline and Loss Function

1) *2-Stage Training*: HBTXR is trained in two stages. Stage 1 focuses on Search-oriented relocalization so that the frame branch learns stable global geometry and objectness. Stage 2 fine-tunes the full hybrid model with Search, Event, and Track supervision jointly, including cross-branch consistency and scheduler-aware quality gating. This stage-wise procedure reflects the distinct optimization characteristics of global relocalization and local residual tracking.

2) *Implicit Network Slimming*: To make the shared transformer and the decoupled heads more deployment-friendly, HBTXR adopts an implicit network slimming view during training. The key idea is to encourage low-importance channels and tokens to collapse through scale regularization rather than to enforce hard pruning during the early design stage. In practice, this can be implemented through sparsity-promoting penalties on normalization scales or channel-gating factors, allowing the final deployment toolflow to map a slimmer but functionally consistent model. The resulting total objective is

$$\mathcal{L} = \lambda_{eye} \mathcal{L}_{eye} + \lambda_{mask} \mathcal{L}_{mask} + \lambda_{search} \mathcal{L}_{search} + \lambda_{event} \mathcal{L}_{event} + \lambda_{track} \mathcal{L}_{track} + \lambda_{ctr} \mathcal{L}_{ctr} + \lambda_{aux} \mathcal{L}_{aux} + \lambda_{slim} \mathcal{L}_{slim}. \quad (6)$$

In addition, HBTXR uses a geometry-aware loss on ellipse center and covariance so that full ellipse shape is supervised rather than only the pupil center.

IV. PROPOSED HBTXR HARDWARE

A. Design Challenges and Co-design Principles

The deployment target of HBTXR is not a generic transformer benchmark but a hybrid eye-tracking workload whose Search and Track modes have fundamentally different behaviors. Search requires refresh-heavy feature regeneration and stronger synchronization across branches, whereas Track benefits from state persistence, feature reuse, and low-overhead incremental update. A uniform hardware path would therefore either waste computation during Track mode or sacrifice robustness during Search mode. This asymmetry motivates an AHCO-style design principle: the hardware should be shared where the computation is dominant, but differentiated where runtime semantics differ.

B. Quantization and Non-linear Approximation

1) *Quantization*: To improve deployment efficiency, HBTXR adopts a mixed-precision quantization strategy for linear layers and memory buffers. The goal is not only to reduce model size, but also to make token projection, MHA, MLP, and head

modules compatible with a streaming FPGA datapath. In an AHCO-style presentation, the quantization policy is selected jointly with the accelerator architecture so that activation ranges, accumulator width, and feature-buffer organization remain consistent across Search and Track modes.

2) *Non-linear Approximation*: Transformer inference includes hardware-unfriendly non-linear operators such as GELU, Softmax, LayerNorm, and ReQuant-style scaling. Following the logic used in hardware-aware transformer designs, HBTXR replaces or approximates these operators with deployment-friendly piecewise-linear or LUT-based implementations. The purpose is to preserve prediction quality while shifting resource demand away from DSP-heavy floating-point processing and toward FPGA-native LUT and lightweight control logic.

C. Accelerator Architecture

1) *Intra-Stage Dataflow*: Within each stage, HBTXR uses dataflows selected according to locality and reuse opportunity. Search-oriented operations favor broader feature regeneration and synchronized token interaction, whereas Track-oriented operations favor residual-state reuse and lightweight head execution. Consequently, the proposed accelerator distinguishes between token-parallel processing for shared backbone stages and stream-oriented state update for Track heads.

2) *Multi-Stage Pipeline*: At the inter-stage level, the accelerator adopts a mixed-granularity pipeline rather than a single coarse or single fine-grained scheme. Globally synchronized stages are executed as phase-level operations, while reuse-friendly stages are executed as streaming tile-level operations. This avoids the excessive buffer cost of a pure coarse-grained pipeline and the incompatibility of a pure fine-grained design with global token dependencies.

3) *Streaming-Oriented Feature Delivery*: Because memory movement is often more expensive than arithmetic in embedded XR deployment, HBTXR employs streaming-oriented feature delivery. Search mode uses refresh buffers to regenerate required shared features, whereas Track mode uses token queues and persistent state buffers so that recently computed context can be reused without re-materializing the entire inference path.

D. Patch Embedding Module

The patch embedding module handles frame and event tokenization before shared-backbone processing. Since frame Search and event Track see different statistics and update frequencies, HBTXR retains modality-specific patch embedding paths. In hardware, this module is responsible for window extraction, patch projection, token packing, and interface normalization before token delivery to the shared backbone engine.

E. Transformer Module

The transformer module forms the dominant compute engine of HBTXR. It executes the shared MHA and MLP blocks for both frame and event tokens. Rather than instantiating disjoint backbones for Search and Track, the hardware maintains one

Figure placeholder
Final artwork pending

Fig. 3. Mode-aware mixed-granularity execution architecture of the proposed HBTXR accelerator. Phase-level execution is used for synchronization-heavy stages, whereas tile-stream execution is used for reuse-friendly update stages.

transformer-oriented compute substrate and reuses it across modes. This shared design is the main reason HBTXR can remain deployment-feasible despite adopting a transformer backbone.

F. Head Module

The head module contains the decoupled Search, Event, Track, Mask, and auxiliary heads. These heads are substantially lighter than the shared backbone, but they are crucial for functional specialization. In hardware, the head module therefore benefits from stream-oriented execution and low-latency output generation. Search-mode heads are activated during refresh, whereas Track-mode heads dominate steady-state operation.

G. CPU–FPGA Workload Partition (Scheduler)

HBTXR adopts a heterogeneous CPU–FPGA organization. The CPU handles frame/event acquisition, threshold comparison, closed-eye handling, state dispatch, and runtime Search/Track scheduling. The FPGA handles patch embedding, shared-backbone inference, decoupled head execution, and mode-specific feature reuse. This partition reflects the fact that control-centric tasks are irregular and mode-dependent, while neural inference stages are regular and well suited to spatial acceleration.

V. EXPERIMENTAL RESULTS

A. Experiments Setup

Dataset: We evaluate HBTXR on a near-eye hybrid eye-tracking benchmark composed of synchronized frame observations, event streams, and dense pupil annotations. Each sample contains a canonicalized frame patch, a polarity-split event tensor, the previous tracking state, and geometry-aware supervision targets. The dataset is split in a subject-disjoint manner for training, validation, and testing.

Software / Hardware: The software model is trained in PyTorch and executed under a host–device heterogeneous runtime. The accelerator-oriented deployment assumes CPU-side scheduling and FPGA-side neural inference. The current JETCAS draft verifies runtime behavior and system organization, while board-level resource and power reporting remain to be finalized.

TABLE I
EXPERIMENTAL SETUP SUMMARY

Item	Setting
Training protocol	Two-stage Search pretrain + hybrid fine-tune
Input frame size	256×256
Input event size	$2 \times 256 \times 256$
Event policy	Fixed-count
Default event count	5000
Accumulation	Fast causal accumulation
Resize policy	Direct square canonical resize
Scheduler signals	Search conf., track conf., similarity, density, eye state
Optimizer	AdamW
Batch size	8
Learning rate	3×10^{-4}
Weight decay	1×10^{-4}
Runtime platform	Host-device heterogeneous execution

Hyper-parameters: The main configuration is summarized in Table I. HBTXR uses two-stage training, 256×256 canonicalized inputs, fixed-count event windows with 5000 events, fast causal accumulation, AdamW optimizer, batch size of 8, learning rate of 3×10^{-4} , and weight decay of 1×10^{-4} .

Workstation: Training and validation are conducted in a standard GPU-based workstation environment, while runtime evaluation follows the host-device execution model described in the hardware section. Since the current source draft does not provide a finalized workstation sentence with exact CPU/GPU specifications, this line is intentionally kept generic and should be replaced by the verified configuration before submission.

B. Proposed HBTXR Evaluation

1) Event-centric vs. Frame-centric vs. Hybrid Accuracy and Latency: Table II reports the operational comparison between the frame-driven Search path, the event-driven Track path, and the scheduled hybrid system. Search mode achieves 0.58 px pupil error, 0.91° gaze error, 97.8% Search success rate, and 0.93 ms latency. Track mode achieves 0.42 px pupil error, 0.68° gaze error, and 0.57 ms latency, corresponding to 1754 Hz effective update rate. Under scheduler-driven hybrid operation, the average runtime becomes 0.63 ms while preserving strong accuracy. We emphasize that the Track row

TABLE II
OPERATIONAL COMPARISON OF FRAME-CENTRIC, EVENT-CENTRIC, AND
HYBRID MODES

Mode	Pupil Err. (px)	Gaze Err. (deg)	Latency (ms)	Rate (Hz)
Frame-driven Search	0.58	0.91	0.93	1075
Event-driven Track	0.42	0.68	0.57	1754
Hybrid scheduled	0.45	0.71	0.63	1587

TABLE III
HARDWARE IMPLEMENTATION RESULTS OF HBTXR

Metric	Search	Track	Scheduled	Note
Latency (ms)	0.93	0.57	0.63	verified in draft
Update rate (Hz)	1075	1754	1587	verified in draft
LUT / DSP / BRAM	TBD	TBD	TBD	post-P&R report
Power	TBD	TBD	TBD	board measurement
Latency breakdown	host / transfer / exec / sync	host / transfer / exec / sync	averaged	insert measured values

should be interpreted as an *event-centric state-conditioned update* rather than a fully stand-alone event-only detector; nevertheless, this operational comparison still makes the Search/Track asymmetry explicit.

2) *Hardware Implementation Results (Resources, Power, Latency Breakdown)*: The verified runtime numbers from the current draft support the latency side of the deployment story, but the source draft does not yet include finalized FPGA resource utilization or measured power. Accordingly, Table III is written as a submission-ready scaffold: the latency figures are preserved, while LUT/DSP/BRAM/Power fields are left for verified board-level insertion. This makes the section structurally complete without fabricating unsupported measurements.

C. Comparison with Other Eye Tracking Algorithm

Table IV summarizes representative prior eye-tracking systems using the metrics reported in their source papers. Because the referenced works differ in task definition, dataset, and runtime platform, the comparison is intentionally source-native rather than forcibly normalized. Even under this conservative presentation, HBTXR stands out by combining sub-millisecond Track latency with both pupil and gaze metrics under one Search/Track framework.

D. Comparison with Other Accelerator

Table V compares HBTXR with representative accelerator-oriented systems across three categories: eye tracking accelerators, transformer accelerators, and object detection / tracking accelerators. EyeCoD represents frame-oriented eye tracking acceleration, SEE-D and JaneEye represent event-centric FPGA/ASIC designs, HG-PIPE is included as a representative ViT accelerator with hybrid-grained pipeline ideas relevant to HBTXR, and AHCO-YOLO is included as the reference style target for algorithm–hardware co-optimization in detection

Figure placeholder
Final artwork pending

Fig. 4. Qualitative Search/Track examples and latency-breakdown placeholder. Final figure panels can combine tracking visualization, scheduler transitions, and measured runtime components.

systems. This comparison clarifies that HBTXR is closest in spirit to AHCO-style co-design: its main novelty is not only low latency, but the explicit coupling of algorithm decomposition and hardware mapping.

E. Ablation Study

Table VI summarizes the ablation study. Removing geometry-stable supervision degrades pupil error from 0.42 to 0.49 pixels and gaze error from 0.68° to 0.80°. Removing decoupled heads increases both error and latency. Disabling the Search/Track scheduler leads to the largest latency increase, from 0.57 ms to 0.71 ms, showing that runtime adaptivity is central to the system value. Removing reuse-oriented execution leaves accuracy unchanged but increases latency to 0.69 ms, showing that feature reuse is primarily a deployment gain.

VI. CONCLUSION

This paper presented HBTXR, an algorithm–hardware co-designed hybrid eye-tracking framework for on-device XR. The main idea is to move beyond undifferentiated multimodal regression and instead treat hybrid eye tracking as a Search/Track system with explicit runtime semantics. Search provides robust frame-guided relocalization, Track provides low-latency event-guided residual update, and the scheduler couples both with deployment-aware hardware execution.

By reorganizing HBTXR in an AHCO-style narrative, this manuscript highlights that the main contribution is not a single model block or a single accelerator trick, but the joint treatment of supervision, transformer design, runtime control, and hardware mapping. Experimental results show that HBTXR achieves 0.42 px pupil error, 0.68° gaze error, and 0.57 ms Track latency while maintaining robust Search recovery. Future work should finalize board-level resource and power measurements, strengthen the quantitative hardware table with post-layout numbers, and further refine the quantization and slimming flow for full implementation closure.

REFERENCES

- [1] A. Plopski, T. Hirzle, N. Norouzi, L. Qian, G. Bruder, and T. Langlotz, “The eye in extended reality: A survey on gaze interaction and eye tracking in head-worn extended reality,” *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.
- [2] H. Kwon, K. Nair, J. Seo, J. Yik, D. Mohapatra, D. Zhan, J. Song, P. Capak, P. Zhang, and P. Vajda *et al.*, “Xrbench: An extended reality machine learning benchmark suite for the metaverse,” *Proceedings of Machine Learning and Systems*, vol. 5, pp. 1–20, 2023.

TABLE IV
COMPARISON WITH REPRESENTATIVE EYE TRACKING ALGORITHMS USING SOURCE-REPORTED METRICS

Work	Mod.	Primary Task	Reported Accuracy	Reported Runtime	Reloc.	Note
Etracker	Frame	Gaze	0.53° gaze accuracy	30–60 Hz	Yes	Mobile near-eye tracker
HE-Tracker	Frame	Gaze	3.655° gaze error	48 fps on AR HMD	Yes	Head-eye coordination model
E-Track	Event	Pupil	3.68 px median error	66.4 ms U-Net inference	No	Event-only detector
FACET	Event	Pupil	0.2030 px PE	0.5302 ms inference	No	Ellipse-based detector
EX-Gaze	Hybrid	Pupil / tracking	1.33 px (saccade), 1.49 px (smooth pursuit)	0.39–0.43 s per 1 s stream on Orin Nano	Yes	2 KHz hybrid system
HBTXR	Hybrid	Pupil + Gaze	0.42 px pupil err.; 0.68° gaze err.	0.57 ms Track mode	Yes	Unified Search/Track system

TABLE V
COMPARISON WITH REPRESENTATIVE ACCELERATORS USING SOURCE-REPORTED METRICS

Category	System	Platform	Reported Throughput / Latency	Reported Efficiency	Positioning w.r.t. HBTXR
Eye tracking	EyeCoD	ASIC-style frame accelerator	240 FPS target throughput	154.32 mW silicon power at 370 MHz	Frame-oriented predict–focus pipeline
Eye tracking	SEE-D	FPGA SoC	0.70 ms latency	2.29 mJ / inference	Event-centric sparse pipeline
Eye tracking	JaneEye	12-nm ASIC	0.5 ms; 2000 FPS	18.9 μ J / frame	Event-centric ASIC co-design
ViT	HG-PIPE	VCK190 FPGA	7118 images / s	381.0 GOPS / W	Hybrid-grained transformer acceleration reference
Obj. det. / tracking	AHCO-YOLO-T	ZCU104 FPGA	79.8 FPS	41.9 FPS / W; 80.5 GOPS / W	AHCO-style algorithm–hardware co-optimization reference
Hybrid eye tracking	HBTXR	CPU–FPGA host-device	0.57 ms Track latency; 1754 Hz	TBD	Runtime-adaptive Search/Track co-design

TABLE VI
ABLATION STUDY OF HBTXR

Variant	Pupil Err. (px)	Gaze Err. (deg)	Latency (ms)
Full HBTXR	0.42	0.68	0.57
w/o geometry-stable supervision	0.49	0.80	0.58
w/o decoupled heads	0.47	0.75	0.62
w/o Search/Track scheduler	0.46	0.73	0.71
w/o reuse-oriented execution	0.42	0.68	0.69

2026.

- [3] L. Chen, Y. Li, X. Bai, X. Wang, Y. Hu, M. Song, L. Xie, Y. Yan, and E. Yin, “Real-time gaze tracking with head-eye coordination for head-mounted displays,” in *Proc. IEEE ISMAR*, 2022, pp. 82–91.
- [4] N. Li, A. Bhat, and A. Raychowdhury, “E-track: Eye tracking with event camera for extended reality applications,” in *Proc. IEEE AICAS*, 2023, pp. 1–5.
- [5] G. Zhao, Y. Yang, J. Liu, N. Chen, Y. Shen, H. Wen, and G. Lan, “EV-Eye: Rethinking high-frequency eye tracking through the lenses of event cameras,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 62169–62182, 2023.
- [6] J. Ding, Z. Wang, C. Gao, M. Liu, and Q. Chen, “FACET: Fast and accurate event-based eye tracking using ellipse modeling for extended reality,” in *Proc. IEEE ICRA*, 2025.
- [7] N. Chen, Y. Shen, T. Zhang, Y. Yang, and H. Wen, “EX-Gaze: High-frequency and low-latency gaze tracking with hybrid event-frame cameras for on-device extended reality,” *IEEE Transactions on Visualization and Computer Graphics*, 2025.
- [8] H. You, C. Wan, Y. Zhao, Z. Yu, Y. Fu, J. Yuan, S. Wu, S. Zhang, Y. Zhang, C. Li *et al.*, “EyeCoD: Eye tracking system acceleration via FlatCam-based algorithm and accelerator co-design,” in *Proc. ISCA*, 2022.
- [9] B. Zhang, Y. Gao, J. Li, and H. K.-H. So, “Co-designing a sub-millisecond latency event-based eye tracking system with submanifold sparse CNN,” in *Proc. CVPR Workshops*, 2024.
- [10] T. Han, A. Li, Q. Chen, and C. Gao, “JaneEye: A 12-nm 2K-FPS 18.9- μ J/frame event-based eye tracking accelerator,” in *Proc. ASP-DAC*, 2026.
- [11] Q. Guo, J. Wan, S. Xu, M. Li, and Y. Wang, “HG-PIPE: Vision transformer acceleration with hybrid-grained pipeline,” 2024.
- [12] J. Kim, J.-K. Kang, and Y. Kim, “AHCO-YOLO: An algorithm–hardware co-optimization framework for energy-efficient and real-time object detection on edge devices,” *IEEE Trans. VLSI Syst.*, 2025.
- [13] C. Lin, J. Hou, and Z. Li, “TimeLens: Event-based video frame interpolation,” *Proc. CVPR*, 2021.
- [14] R. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. Berg, W.-Y. Lo, *et al.*, “Segment anything,” *Proc. ICCV*, 2023.
- [15] A. Kirillov *et al.*, “Grounded-SAM: Marrying grounding DINO with segment anything,” 2023.